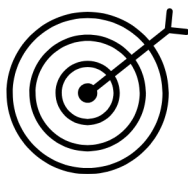




AULA 3

Processamento de linguagem natural



Objetivos de aprendizagem da aula

Ao final desta aula, você irá:

- Compreender os conceitos básicos de processamento de linguagem natural e processamento de texto (*word embeddings*, *one-hot encoding*, *bag of words*, TF-IDF).
- Aprender técnicas para preparação dos dados.
- Compreender o pré-processamento e a tokenização.
- Entender conceitos de similaridade de documentos, análise de sentimentos, reconhecimento de entidades.

Que história é essa?

Nesta terceira aula, focaremos em aplicações para processamento de linguagem natural. Em especial, apresentaremos aplicações envolvendo limpeza e pré-processamento de dados, uma breve introdução ao processamento de linguagem natural, similaridade de documentos e reconhecimento de entidades nomeadas.

Processamento de linguagem natural

O processamento de linguagem natural - PLN, é um ramo da inteligência artificial que lida com a análise, a compreensão e a geração de padrões de linguagens que os seres humanos usam

naturalmente para se expressar e interagir, seja da forma escrita ou falada.

Por 'linguagem natural' queremos dizer os idiomas que são usados para as comunicações do dia a dia dos seres humanos, como inglês ou português. Diferentemente das linguagens artificiais, como linguagens de programação e notações matemáticas, as linguagens naturais evoluem à medida que passam de geração a geração, e é difícil descrevê-las completamente por meio de regras explícitas.



As tecnologias baseadas em PLN estão se tornando cada vez mais difundidas. Por exemplo, telefones e *handhelds* suportam previsão de texto e reconhecimento de escrita; motores de busca para a *web* permitem acessar informações contidas em textos não estruturados; os sistemas de tradução automática permitem obter textos escritos em chinês e lê-los em espanhol.

Ao fornecer interfaces homem-máquina mais naturais, bem como acesso mais sofisticado às informações armazenadas, o processamento de linguagem passou a desempenhar um papel central na sociedade da informação multilíngue.

Nesse sentido, podemos dividir o processamento de linguagem natural em três vertentes ao longo de sua história: uma onda racionalista de desenvolvimento, uma empírica e outra baseada em redes neurais.

Racionalista



Para estruturar o conhecimento na linguagem humana, a vertente racionalista se baseia em uma estrutura fixa, ou seja, um conjunto de regras bem definidas. Um especialista as define e tenta aplicá-las ao processamento de informação. Esse modelo baseado em regras foi um consenso dominante na área entre os anos de 1960 e 1980. Contudo, é impossível mapear todo o conjunto de regras que definem ações e conceitos da linguagem humana, visto que ela mesma não é fixa, e sim mutável.

Empírica



É baseada na extração de *features* de um conjunto de dados. Esse modelo leva em conta um conceito chamado “engenharia de *features*”, em que estruturas relevantes de um conjunto de informações são definidas e processadas, muitas vezes manualmente por um especialista humano, e, depois, processadas por algum algoritmo inteligente. Esse tipo de sistema era focado em aplicações específicas de um domínio, em que se encontram sistemas de aprendizado baseados em casos de aplicação, e não generalistas.

Baseada em redes neurais

↓

Esta é uma vertente moderna de PLN baseada no aprendizado de grandes volumes de dados. Esses volumes são chamados de *corpora* ou *corpus* textuais. Através de algoritmos de mineração extraindo texto, fazemos seu pré-processamento em *tokens* e os enviamos como entrada a algoritmos de redes neurais que tentam generalizar a compreensão de estruturas textuais. Nesse sentido, algoritmos modernos de PLN dispensam a necessidade de um especialista para extrair *features* manualmente. Porém, é preciso dizer que mesmo algoritmos modernos de PLN precisam passar, após seu treinamento, por validação humana.

Na próxima seção, conheça os conceitos a respeito do processamento de texto em português.

Tarefas de processamento de linguagem natural

A língua portuguesa, e em especial línguas de forte raiz latina, apresentam estruturas gramaticais e semânticas muito mais complexas do que o inglês, segundo muitos linguistas. Isso é facilmente notado em vários sistemas de PLN, como, por exemplo, os tradutores automáticos, visto que em muitos casos essa língua é muito mais desafiadora para ser interpretada.

Nesse sentido, em nossos exemplos, usaremos o **Natural Language Toolkit (NLTK)**, uma plataforma em Python para processamento de linguagem humana. O NLTK também oferece diversos *datasets* textuais para exercício e experimentação, sendo que, nesta aula, vamos trabalhar com o *dataset* sobre a obra de Machado de Assis. Para isso, precisamos, primeiramente, importar o NLTK em nosso código Python e fazer o *download* do *dataset*.

```
import nltk
nltk.download('machado')

from nltk.corpus import machado

print(machado.fileids())
```

Fileids representam o nome e o *path* de onde foram salvos os arquivos do nosso *dataset*.

Limpeza de dados

No processo de limpeza de dados, **removemos caracteres especiais, símbolos e pontuação dos dados brutos que não contêm informações estruturadas para um modelo aprender** e podem ser considerados apenas ruídos por nossos modelos de PLN. É claro que esse processo também depende do meio em que estamos trabalhando. Por exemplo, se queremos tratar informações relevantes, como problemas de economia ou computação, dependendo do texto com o qual trabalhamos, alguns caracteres especiais precisam ser mantidos, como, por exemplo, \$ e vírgulas. De maneira geral, na maioria dos casos, esses termos são removidos.

No exemplo a seguir, vamos usar expressões regulares para remover caracteres especiais.

```
cod.py

import re

#importar o texto do dataset
text = machado.raw('contos/macn081.txt')

cleaned_text = re.sub('[^A-Za-z0-9]+', ' ', text) #remover caracteres especiais

print(cleaned_text)
```

Pré-processamento de dados

Esta etapa envolve **transformar os dados que foram limpos, mas ainda próximos do seu estrado bruto, em informações estruturadas**. Esse processo começa convertendo todas as letras para minúsculas. Esta é uma das formas mais simples e eficazes de pré-processamento de texto, pois ajuda a remover ambiguidades. Por exemplo, sem executar essa tarefa, uma máquina poderia considerar as palavras “Brasil”, “brasil” e “BRASIL” como instâncias diferentes.

Outra etapa principal é a **tokenização**, que nada mais é do que quebrar nosso conjunto de texto em *tokens* individuais. Para a tokenização, precisamos fazer o *download* do módulo *punkt*. Esse tokenizador divide um texto em uma lista de frases usando um algoritmo não supervisionado para construir um modelo para palavras abreviadas, colocações e palavras.

```
cod.py

cleaned_text = cleaned_text.lower()
from nltk.tokenize import word_tokenize
nltk.download('punkt')
tokens = nltk.word_tokenize(cleaned_text)

print(cleaned_text)
print(tokens)
```

Stop-words

Stop-words são palavras comuns que não contribuem com muita informação em um documento de texto. Palavras como “o”, “é” e “um” têm menos valor e adicionam ruído aos dados do texto.

Há uma lista de palavras irrelevantes embutida no NLTK que podemos usar para remover palavras de documentos de texto. No entanto, esta não é a lista de palavras irrelevantes padrão para todos os problemas. Também podemos definir nosso próprio conjunto de palavras com base no domínio.

```
cod.py

nltk.download('stopwords')
from nltk.corpus import stopwords

stop_words = stopwords.words('portuguese')

filtered_review = [word for word in tokens if word not in stop_words] # removing stop words

print(filtered_review)
```

Stemming

Stemming é o processo de reduzir uma palavra à sua palavra raiz/raiz. Reduz a inflexão nas palavras (por exemplo, “ajuda”, “ajudando”, “ajudou”, “ajudado”) à sua forma raiz (por exemplo, “ajuda”). Ele remove os afixos morfológicos das palavras, deixando apenas o radical da palavra.

```
cod.py

nltk.download('rslp')
stemmer = stemmer = nltk.stem.RSLPStemmer()

stemmed_review = [stemmer.stem(word) for word in filtered_review]

print(stemmed_review)
```

FreqDist

Podemos ver também a frequência com que as palavras se repetem.

```
cod.py

from nltk.probability import FreqDist

fd = FreqDist(filtered_review)
for word in list(fd.keys())[:20]:
    print(word, fd[word])
```

N-grams

N-grams são a combinação de várias palavras usadas juntas. *N-grams* com N = 1 são chamados de unigramas; da mesma forma, bigramas (N = 2), trigramas (N = 3), e assim por diante.

N-grams podem ser usados quando queremos preservar informações de sequência no documento, como qual palavra provavelmente seguirá a dada. Os unigramas não contêm nenhuma informação de sequência porque cada palavra é tomada individualmente.

```
cod.py

from nltk import ngrams
target_word = 'janeiro'
fd = FreqDist(ng
    for ng in ngrams(text, 5)
    if target_word in ng)
```

Convertendo texto

Agora, após o pré-processamento de texto, precisamos representar numericamente os dados, ou seja, codificar os dados em números que podem ser usados por algoritmos. Para isso, podemos criar uma

bag of words (BOW). Ela é uma das técnicas de vetorização de texto mais simples. A intuição por trás da BOW é que duas sentenças são ditas semelhantes se contiverem um conjunto de palavras semelhante.

É um algoritmo que transforma o texto em **vetores de tamanho fixo**. Isso é possível contando o número de vezes que a palavra está presente em um documento. As ocorrências de palavras permitem comparar diferentes documentos e avaliar suas semelhanças para aplicações, como pesquisa, classificação de documentos e modelagem de tópicos.

```
cod.py

import pandas as pd
import numpy as np
import collections

doc1 = 'Data Science é minha área preferida!'
doc2 = 'Data Science é essencial para o sucesso da empresa!'
doc3 = 'Eu não gosto da área de Data Science'

l_doc1 = re.sub(r"[^a-zA-Z0-9]", " ", doc1.lower()).split()
l_doc2 = re.sub(r"[^a-zA-Z0-9]", " ", doc2.lower()).split()
l_doc3 = re.sub(r"[^a-zA-Z0-9]", " ", doc3.lower()).split()

from sklearn.feature_extraction.text import CountVectorizer
vectorizer = CountVectorizer()

X = vectorizer.fit_transform([doc1,doc2,doc3])
df_bow_sklearn = pd.DataFrame(X.toarray(),columns=vectorizer.get_feature_names())
print(df_bow_sklearn.head())
```

Term-frequency

O número de vezes que uma palavra aparece em um documento dividido pelo número total de palavras no documento. Cada documento tem sua própria frequência relativa a cada termo. Vamos considerar a *bag of words* como um dicionário:

```
cod.py

def computeTF(bagOfWords):
    tfDict = {}
    bagOfWordsCount = len(bagOfWords)
    for word, count in bagOfWords.items():
        tfDict[word] = count/float(bagOfWordsCount)
    return tfDict
```

Inverse term-frequency

O log do número de documentos dividido pelo número de documentos que contêm a palavra “w”. A frequência de dados inversa determina o peso de palavras raras em todos os documentos do *corpus*.

```
cod.py

def computeIDF(documents):
    import math
    N = len(documents)

    idfDict = dict.fromkeys(documents[0].keys(), 0)
    for document in documents:
        for word, val in document.items():
            if val > 0:
                idfDict[word] += 1

    for word, val in idfDict.items():
        idfDict[word] = math.log(N / float(val))
    return idfDict
```

Etiquetagem de classe gramatical (*POS tagging*)

A *POS tagging* é a tarefa de determinar a classe gramatical de cada palavra em uma sentença. Esta é uma tarefa extremamente importante em ferramentas para tradução automática ou reconhecimento de fala, por exemplo. No próximo exemplo de código, usaremos o pacote SpaCy em Python, que faz o processamento avançado de linguagem natural em Python.

```
import spacy
# install Portuguese model language
import spacy.cli
spacy.cli.download("pt_core_news_sm")
nlp = spacy.load('pt_core_news_sm')

doc = nlp('Este é um exemplo do curso de Data Science')

for token in doc:
    print(token.text, token.pos, token.pos_)
```

Dependency parsing

A ideia é descobrir dependências entre os termos de uma sentença. Na prática, é a estrutura sintática que consiste em relações binárias, assimétricas, entre palavras de uma sentença. Construímos uma árvore exemplificando essas relações, que pode ser usada em sistemas inteligentes para extrair expressões lógicas e disparar ações.

```
import spacy
# install Portuguese model language
import spacy.cli
spacy.cli.download("pt_core_news_sm")
nlp = spacy.load('pt_core_news_sm')

doc = nlp('Este é um exemplo do curso de Data Science')

for token in doc:
    print(token.text, token.pos, token.pos_)

from spacy import displacy
from IPython.display import SVG, display
def showSVG(s):
    display(SVG(s))

graph01 = displacy.render(doc)
showSVG(graph01)
```

A figura a seguir apresenta o *output* gerado pelo código apresentado anteriormente.

Interativo

Interativo apresentando uma caixa azul em fundo branco, com o texto: Este é um exemplo do curso de data Science. Abaixo, lemos: As tags das etiquetas. No slide seguinte, lê-se: Esta figura apresenta cada uma das dependências e estruturas encontradas na frase. Abaixo, vemos a frase abre aspas este é um exemplo do curso de data science fecha aspas. Ela é apresentada com setas explicativas ligadas a cada um dos elementos. As setas representam a dependência entre as palavras e os modificadores entre elas: Este: PRON abre parênteses nsubj fecha parênteses; é: AUX abre parênteses cop fecha parênteses; um: DET; exemplo: NOUN; do: ADP abre parênteses case, nmod fecha parênteses; curso: NOUN; de: ADP abre parênteses case, nmod fecha parênteses; data: PROPN; Science: PROPN abre parênteses flat dois pontos name fecha parênteses. No slide seguinte, vemos o texto: as setas representam as dependências entre as palavras e os modificadores entre elas. Abaixo, vemos a mesma frase do slide anterior, apresentando a relação entre cada palavra através de setas. A palavra abre aspas exemplo fecha aspas está ligada a abre aspas este fecha aspas por uma relação nsubj, a abre aspas é fecha aspas por uma relação cop, a abre aspas um fecha aspas por uma relação det e a abre aspas curso fecha aspas por uma relação nmod. A palavra abre aspas curso fecha aspas está ligada a abre aspas do fecha aspas por uma relação case e a abre aspas data fecha aspas por uma ligação nmod. A palavra abre aspas data fecha aspas está ligada a abre aspas de fecha aspas por uma relação case e a abre aspas science fecha aspas por uma relação flat dois pontos name. No slide seguinte, vemos o texto: As tags de cada palavra a seguir apresentam as tags geradas por uma etiquetagem de classes gramaticais, que podem incluir o seguinte. Aqui, vemos uma tabela explicativa apresentando cada tag e suas classes gramaticais: ADJ: adjetivo, por exemplo, grande, velho, verde, incompreensível, primeiro ADP: aposição, por exemplo, em, para, durante. ADV: advérbio, por exemplo, muito, amanhã, para baixo, onde, lá. AUX: auxiliar, por exemplo, é abre parênteses tem feito fecha parênteses, vai abre parênteses fazer fecha parênteses, deveria abre parênteses fazer fecha parênteses. CONJ: conjunção, por exemplo, e, ou, mas. CCONJ: conjunção coordenativa, por exemplo, e, ou, mas. DET: determinante, por exemplo, um, o. INTJ: interjeição, por exemplo, psst, ai, bravo, olá. NOUN: substantivo, por exemplo, menina, gato, árvore, ar, beleza. NUM: numeral, por exemplo, 1, 2017 (numerais), um, setenta e sete (por extenso), IV, MMXIV (números romanos). PART: partícula, por exemplo, apóstrofo s. PRON: pronome, por exemplo, eu, você, ele, ela, eu, eles mesmos, alguém. PROPN: nome próprio, por exemplo, Mary, John, Londres, OTAN, HBO. PUNCT: pontuação, por exemplo, abre parênteses, fecha parênteses, interrogação. SCONJ: conjunção subordinativa, por exemplo, se, enquanto, que. SYM: símbolo, por exemplo, cifrão, por cento, arroba, mais, menos, asterisco, igual a, fecha parênteses, emoji mostrando a língua. VERB: verbo, por exemplo, corre, correu, correndo, come, comeu, comendo X: outros. ESPAÇO: espaço.



Aprenda mais

Dependency parsing

Clique aqui e acesse um material sobre *dependency parsing* para saber mais.

Word embedding

Antes de entrarmos em similaridade de documentos, precisamos compreender o que são *embeddings* de palavras. **Word embeddings é uma técnica em que palavras são transformadas em uma representação na forma de um vetor numérico.** Esses vetores tentam capturar várias características de uma palavra em relação ao texto geral. Essas características podem incluir a relação semântica da palavra, definições e contexto. A motivação para representar palavras na forma de vetores em PLN é que algoritmos de aprendizado de máquina não são capazes de processar informação de texto bruto, apenas a sua representação numérica.

A representação mais simples é na forma de **one-hot encoding**, em que os dados são mapeados para vetores, cada um representando uma categoria. Por exemplo:

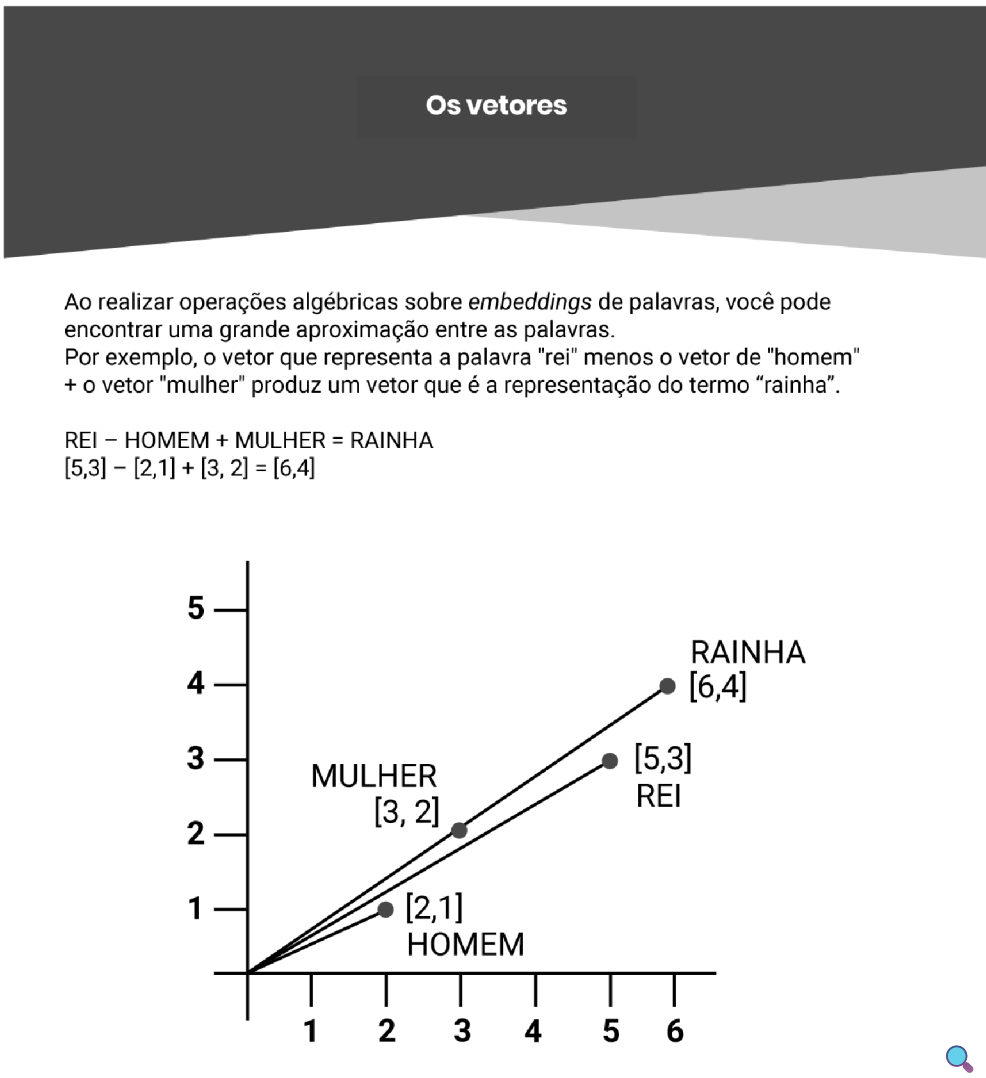
```

cod.py
Casa = [1, 0, 0, 0, 0, 0, ... 0]
carro = [0, 1, 0, 0, 0, 0, ... 0]
gato = [0, 0, 1, 0, 0, 0, ... 0]
cachorro = [0, 0, 0, 1, 0, 0, ... 0]

```

Word2Vec

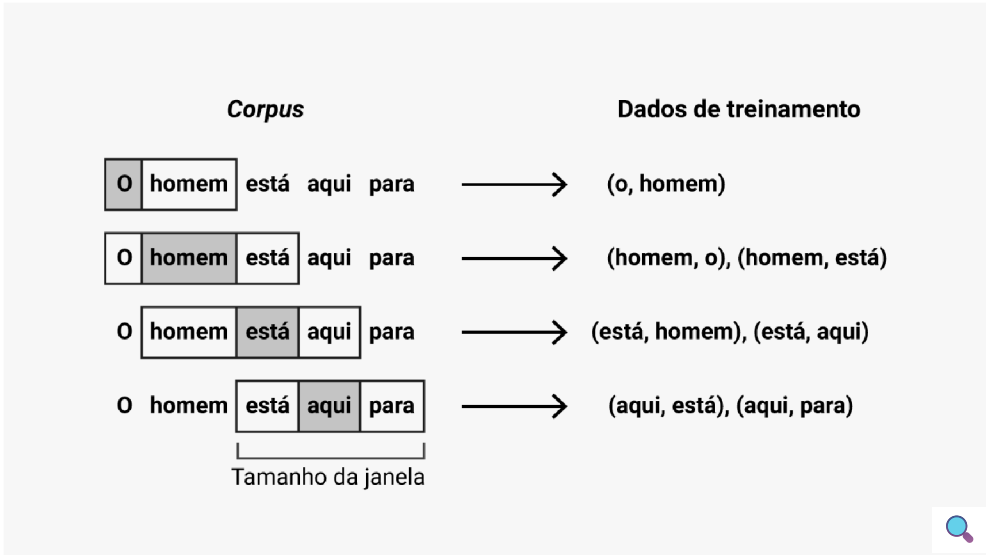
A eficácia do Word2Vec vem de sua capacidade de **agrupar vetores de palavras semelhantes e realizar cálculos sobre eles** que têm uma relação de semântica entre as palavras. Dado um conjunto de dados grande o suficiente, o Word2Vec pode fazer estimativas sobre o significado de uma palavra com base em suas ocorrências no texto. Essas estimativas geram associações de palavras com outras palavras do *corpus*. Por exemplo, palavras como “rei” e “rainha” são muito semelhantes entre si no contexto semântico.



Como podemos perceber no exemplo anterior, esses vetores não são na verdade gerados por uma representação *one-hot encoding*. Então como esses vetores são gerados nesse caso?

O Word2Vec é na verdade uma **rede neural que utiliza uma técnica chamada skip-gram**. O modelo *skip-gram* é um algoritmo simples com uma camada oculta treinada para prever a probabilidade de determinada palavra estar presente próximo a outra palavra dada como entrada considerando um *corpus* de texto, ou seja, calcula a probabilidade de palavras ocorrerem em uma sentença e próximo umas das outras.

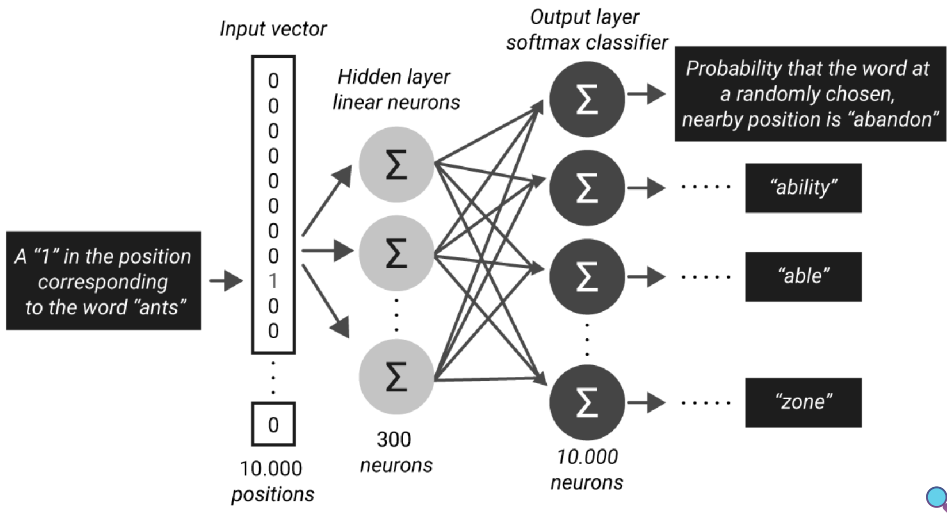
Nessa arquitetura, a palavra de entrada é analisada, e o modelo tenta prever as palavras que normalmente ocorrem próximo a ela antes e depois dessa palavra atual dada uma janela de “n” palavras. Esse modelo essencialmente tenta aprender e prever as palavras de contexto em torno da palavra de entrada especificada. O processo pode ser descrito visualmente conforme é mostrado a seguir:



Como visto no processo anterior, dado algum *corpus* de texto, uma palavra-alvo é selecionada em, considerando aqui neste exemplo, uma janela de uma palavra antes e depois. Os dados de treinamento consistem em combinações dos pares dessa palavra-alvo e todas as outras palavras na janela. Estes são os dados de treinamento resultantes para uma rede neural.

Uma vez que o modelo é treinado, podemos essencialmente produzir uma probabilidade de uma palavra aparecer próximo à palavra-alvo

no *corpus* de texto analisado. A imagem a seguir representa a arquitetura da rede neural para o modelo *skip-gram*:



Um *corpus* pode ser representado como um vetor de tamanho N, em que cada elemento em N corresponde a uma palavra do *corpus*. Durante o processo de treinamento, temos um par de palavras-alvo e de contexto. O *array* de entrada terá 0 em todos os elementos, exceto para a palavra-alvo. A palavra-alvo será igual a 1.



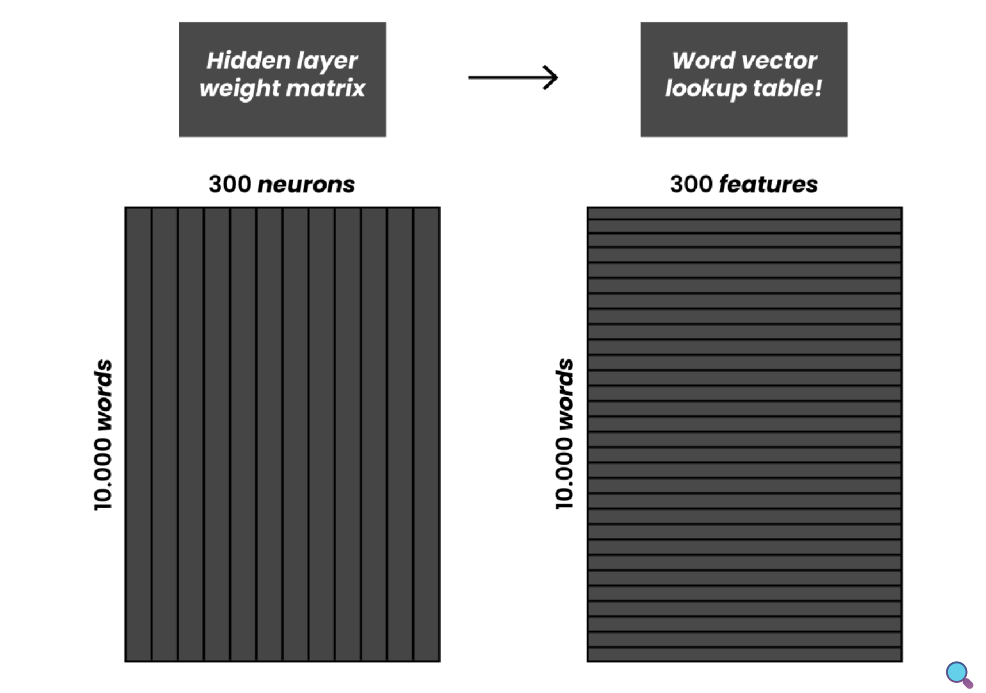
A camada oculta aprenderá a representação *embedding* de cada palavra, produzindo um espaço de *embedding* d-dimensional. A camada de saída é uma camada densa com uma função de ativação *softmax*. A camada de saída produzirá essencialmente um vetor do mesmo tamanho que a entrada, e cada elemento no vetor consistirá em uma probabilidade. Essa probabilidade indica a semelhança entre a palavra-alvo e a palavra associada no *corpus*.

Contudo, aqui está a genialidade desse modelo de rede neural: o objetivo de nosso treinamento não é a saída de nossa rede neural, e sim os pesos da camada interna. Isso nos permite criar vetores que na verdade são uma representação densa das palavras do *corpus* de texto analisado.

Imagine que temos um conjunto de dados com centenas de milhares de palavras e outros termos.

Uma representação tradicional poderia ser impraticável considerando o processamento necessário para analisar vetores de probabilidade gigantescos. Agora, imagine que a camada interna da rede tenha 300 neurônios.

Ao final do processo de treinamento, podemos usar cada linha da matriz de pesos como um vetor associado a cada palavra. Dessa forma, teremos vetores densos e compactos com os quais poderemos realizar nossas operações de forma eficiente.



Outra vantagem dessa estratégia é a possibilidade de gerarmos visualizações baseadas nas informações aprendidas. **Podemos aplicar sobre esses vetores algoritmos para redução de dimensionalidade e os projetos em visualizações 2D.**

Isso pode nos ajudar a criar ferramentas para nos dar *insights* a respeito de nosso conjunto de informações e ser uma grande ferramenta de auxílio para cientista de dados em suas análises. Um exemplo de uma visualização gerada dessa forma pode ser visto na figura a seguir:



Aprenda mais

O TensorFlow disponibiliza essa ferramenta exploratória. Podemos fazer o *upload* de nossos dados e visualizá-los. [Clique aqui](#) para acessar a ferramenta.

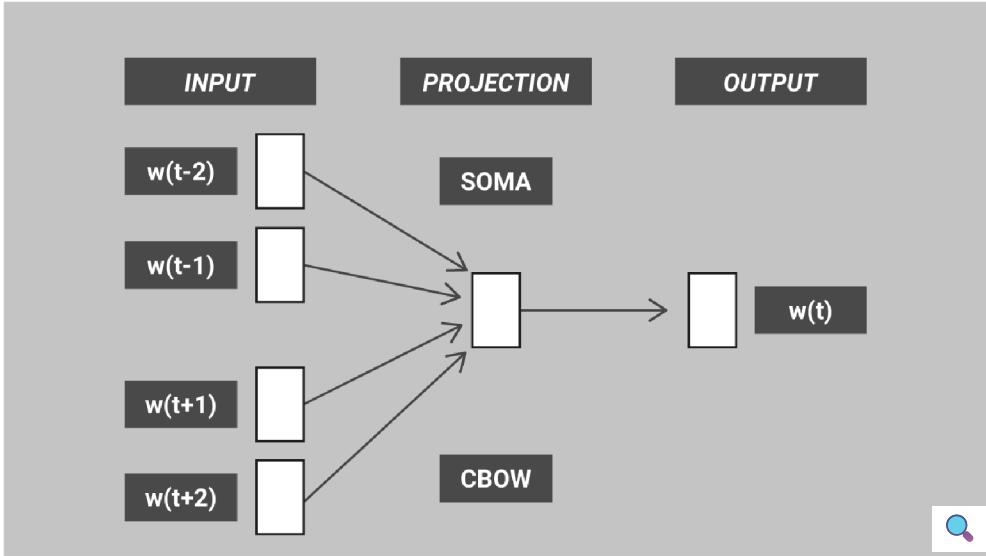
Doc2Vec

Agora que compreendemos como o Word2vec funciona, não é difícil compreender a sua evolução, o Doc2Vec. **O objetivo do Doc2vec é criar uma representação numérica de um documento, independentemente de seu tamanho.** No entanto, diferentemente das palavras, os documentos são agrupados não apenas por questões semânticas, mas também por conceitos, e não há uma forma estruturada de representá-los diretamente.

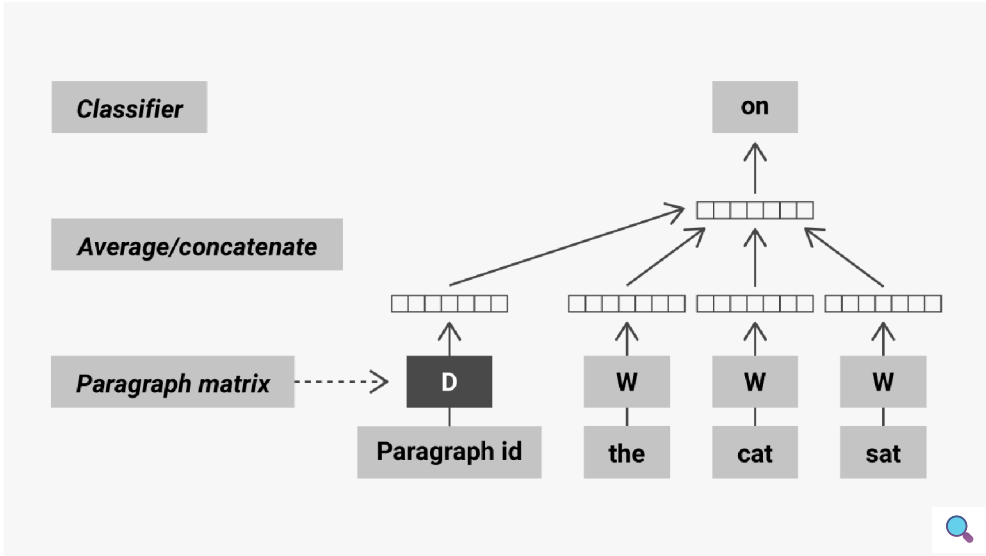
O conceito do Doc2Vec é parecido com o do algoritmo anterior, porém adicionamos um campo a mais, que representa o ID de um documento, ou seja, existe um vetor associado que representa esse identificador.

O modelo do Doc2Vec tem duas variantes, mas a que normalmente tem melhores resultados é uma extensão do **Continuous Bag of Words (CBOW)**. Esse algoritmo tenta essencialmente prever uma palavra a partir de uma lista de palavras de contexto. Isto é, dada uma amostra de palavras, ele tenta prever qual palavra do nosso *corpus* tem a maior probabilidade de estar associada a essa lista.

A intuição por trás desse modelo é simples: dada uma frase 'eu adoro *data science*', escolheremos nossa palavra-alvo como “adoro” e uma lista de palavras relacionadas, como [“eu”, “*data*”, “*science*”]. O que esse modelo fará é pegar as representações distribuídas das palavras de contexto para tentar prever a palavra-alvo “adoro”.



No Doc2Vec, em vez de usarmos apenas palavras para prever a próxima palavra, também adicionamos outro vetor de *features*, que é exclusivo para cada documento. Assim, ao treinar os vetores de palavras, o vetor de documento D também é treinado, e, ao final do treinamento, temos da mesma forma uma representação densa do documento.



O modelo apresentado é chamado de **Distributed Memory version of Paragraph Vector (PV-DM)**. Ele atua como uma espécie de memória que armazena o tópico do documento. **Enquanto os vetores de palavras representam o conceito de uma palavra, o vetor de documentos pretende representar o conceito de um documento.** A rede neural Doc2vec pode ser treinada da seguinte forma: é necessário um conjunto de documentos em que um vetor de palavras W é gerado para cada palavra e um vetor de documento D é gerado para cada documento.

O modelo também treina pesos para uma camada oculta com uma função de ativação *softmax*, responsável pelos valores de probabilidade na saída da rede. Na etapa de inferência, um novo documento pode ser apresentado, e todos os pesos são fixados para calcular o vetor do novo documento com base nas amostras do *corpus* que foi treinado.



Reflita

Vamos tomar, por exemplo, a análise de similaridade de documentos e instituições que demandam sempre uma análise de grandes volumes de informação, como é o caso do campo jurídico.

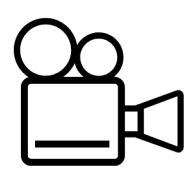
Imagine que podemos usar a técnica de Doc2Vec vista anteriormente com um meio de auxílio nesse processo. Podemos gerar vetores de documentos para todo o nosso conjunto de dados e, com isso, geraremos *embeddings* para os documentos. Podemos calcular a similaridade entre essas *embeddings*. Como são vetores numéricos, a similaridade do cosseno para analisar a semelhança de cada documento com os demais é uma escolha natural.

Agora imagine ainda expandir esse contexto e criar ferramentas que tornem possível, por meio da similaridade de documentos, criar aplicações para magistrados e advogados. Podemos criar ferramentas que podem sugerir processos com históricos semelhantes a ajudar na análise de jurisprudência.

Contudo, ficam aqui dois questionamentos:

1. Quais desafios você vê ao propor tecnologias como esta?
2. Na sua opinião, quais são as implicações éticas desse tipo de análise?

Agora que você já refletiu, vamos em frente!



Processamento de texto e redes neurais para PLN

Neste vídeo, você irá aprender a fazer a limpeza de dados de texto, *bag of words* e TF-IDF. Ainda, irá revisar os conceitos de redes neurais da seção anterior e conhecer uma nova arquitetura, chamada LSTM.

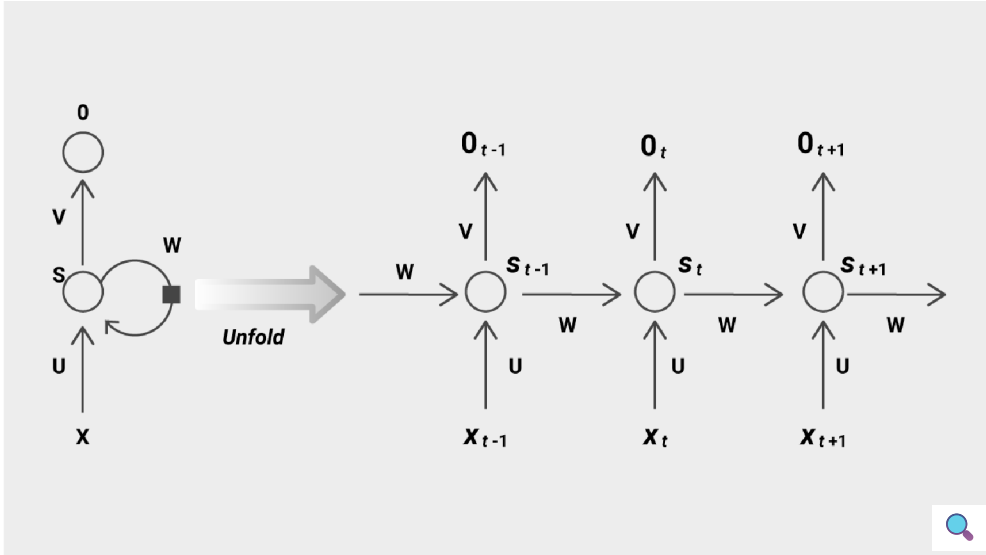


Uma rede neural recorrente (RNN) é uma rede neural especializada em processar uma sequência de dados. Para tarefas que envolvem entradas sequenciais, como fala e linguagem, é indicado usar redes neurais baseadas nessa arquitetura. Atualmente existem versões mais modernas de redes neurais para processamento de texto além das recorrentes. Porém, a fim de introduzir esse assunto, vamos focar nesse exemplo, por ser uma arquitetura mais simples e de fácil implementação.

As RNNs são chamadas de “recorrentes” porque realizam a mesma tarefa para cada elemento de uma sequência, com a saída sendo dependente dos cálculos anteriores. Outra maneira de pensar sobre RNNs é que elas têm uma “memória” que captura informações sobre o que foi calculado até agora. Em um problema de PLN, como um sistema de autocomplete, esse tipo de rede é uma das estratégias usadas porque é possível fazer com que ele aprenda sequências de palavras e force seus estados internos a produzir uma espécie de memória que irá sugerir uma palavra indicada.

Conceitualmente, a RNN difere de uma rede neural padrão, pois a entrada padrão em uma RNN é uma palavra em vez de toda a amostra, como no caso de uma rede neural padrão. Isso dá flexibilidade para a rede trabalhar com comprimentos variados de sentenças, algo que não pode ser alcançado em uma rede neural padrão devido à sua estrutura fixa. Ela também oferece uma vantagem adicional de compartilhar *features* aprendidas em diferentes posições de uma sentença no texto e que não podem ser obtidas em uma rede neural padrão.

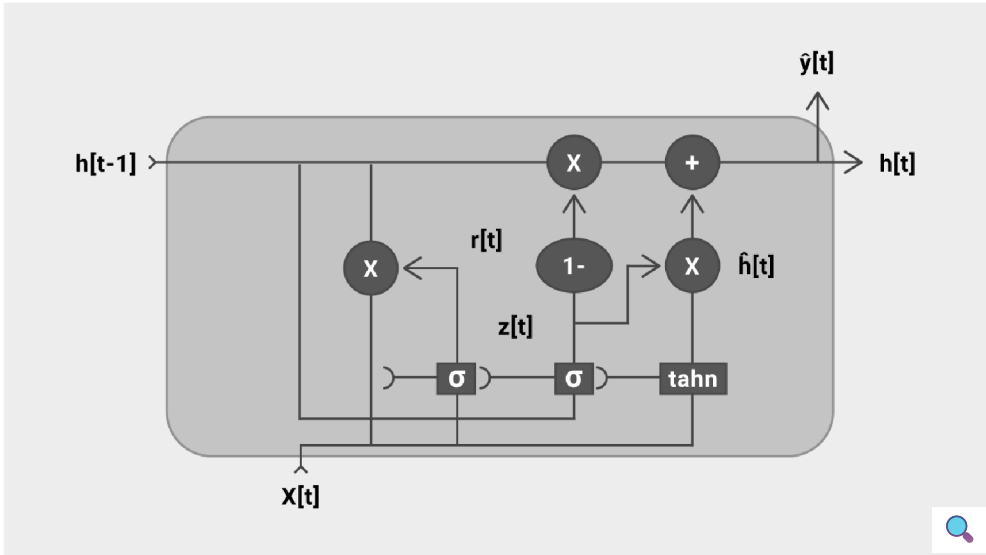
Uma RNN trata cada palavra de uma sentença como uma entrada separada em um instante no tempo “t” e usa o resultado da ativação da saída em “t-1” também como entrada, além da entrada atual no tempo “t”. A ideia aqui é forçar a rede a aprender estados intermediários e ter uma espécie de memória que analisa sequências de texto. O diagrama a seguir mostra uma estrutura detalhada de uma arquitetura RNN.

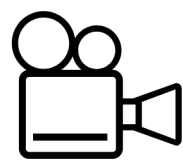


As RNNs têm algumas limitações principais, entre as quais as citadas a seguir:

- **RNNs** são capazes de capturar as dependências em apenas uma direção da linguagem, isto é, basicamente, assume-se que uma palavra que vem depois da atual não tem efeito sobre o significado da palavra analisada. Contudo, mesmo que não tenhamos experiência em análise textual, semântica e gramática, sabemos que isso não representa o mundo real.
- As **RNNs** também não conseguem capturar dependências ditas como “longas”, isto é, estruturas frasais que, apesar de dependentes no contexto sintático, não aparecem próximas.
- Elas ainda têm o problema chamado **vanishing gradient**, um problema clássico de redes neurais em que, durante o treinamento, o cálculo do gradiente se torna muito pequeno, impedindo efetivamente que os pesos da rede sejam alterados; consequentemente, a rede neural “trava” seu aprendizado.

É uma modificação na unidade recorrente básica que ajuda a capturar dependências ditas longas e corrige o problema do *vanishing gradient*. Uma *gated recurrent unit* (GRU) consiste em uma unidade de memória adicional comumente chamada de *update gate*. Além da arquitetura usual com *sigmoide* e *softmax*, contém uma unidade adicional com uma função tanh. A função tanh é usada porque sua saída pode ser positiva e negativa, entre -1 e 1 , portanto, pode ser usada para aumentar e diminuir uma escala. A saída dessa unidade é, então, combinada com a entrada para atualizar o valor da célula de memória.





Similaridade de documentos

Neste vídeo, você vai estudar códigos em Python e conhecer o passo a passo de cada bloco. O *link* desse código será disponibilizado para você praticar depois na Técnica Aplicada.



E agora acompanhe nosso convidado Jonatas Grosman, pesquisador da PUC-Rio comentando sobre sua experiência.



React de mercado

Speech to text e o desafio da Hugging Face

Acompanhe o professor convidado Jonatas Grosman e entenda mais sobre o reconhecimento de fala e o processamento de textos.



Referências

Clique aqui para acessar as referências e créditos desta aula.



DHAR, A.; MUKHERJEE, H.; DASH, N. S.; ROY, K. Text categorization: past and present. **Artificial Intelligence Review**, v. 54, p. 3007-3054, 2021.

GRAVES, A. Long short-term memory. *In*: GRAVES, A. **Supervised sequence labelling with recurrent neural networks**. [s.l.]: Springer, 2012. p. 37-45.

GRAVES, A.; MOHAMED, A.; HINTON, G. Speech recognition with deep recurrent neural networks. *In*: 38th INTERNATIONAL CONFERENCE ON ACOUSTICS, SPEECH AND SIGNAL

PROCESSING, 2013, Vancouver. **Proceedings** [...]. Vancouver, 2013.

LE, Q. V.; MIKLOV, T. Distributed representations of sentences and documents. **arXiv**, 2014.

MIKLOV, T.; CHEN, K.; CORRADO, G. S.; DEAN, J. Efficient estimation of word representations in vector space. **arXiv**, 2013.

SETHY, A.; RAMABHADHAN, B. Bag-of-word normalized n-gram models. *In*: 9th ANNUAL CONFERENCE OF THE INTERNATIONAL SPEECH COMMUNICATION ASSOCIATION, Brisbane, 2008. **Proceedings** [...]. Brisbane, 2008.

TF-IDF Kaggle. Disponível em:
<https://www.kaggle.com/code/edchen/tf-idf>. Acesso em: 12 fev. 2023.

THANAKI, J. **Python natural language processing**. [s.l.]: Packt Publishing Ltd., 2017.

Banco de imagens Pexels, Pixabay, Envato, Freepik e Shutterstock.